60 FPS 4D Gaussian Splatting in a Chrome Tab

How a moving capture becomes something you can open

Mind In a Device

Monthly Tech Talk

October 2026

Why this talk

We have been looking at Gaussian splatting as a reconstruction method. I write JavaScript, so my question starts after the fit exists.

A moving scene still has to reach someone whose machine has a browser.

Rust writes that file once. JavaScript is the program that plays it.

Where this hour goes

1. The 3D Gaussian representation, and what a time axis does to it.
2. Why a folder of raw frames misses a browser.
3. The `.splat4d` file, and which language plays it back.
4. One scene, on this laptop.

3D Gaussian splatting

Kerbl et al., SIGGRAPH 2023. A scene is a set of anisotropic Gaussians, fit to calibrated images.

$$G_i(\mathbf{x}) = \exp\Big(-\tfrac{1}{2}(\mathbf{x}-\mu_i)^\top \Sigma_i^{-1}(\mathbf{x}-\mu_i)\Big), \qquad \Sigma_i = R_i S_i S_i^\top R_i^\top .$$

Projected into the image, and composited in depth order:

$$C(\mathbf{p}) = \sum_{i \in \mathcal{N}} c_i \, \alpha_i(\mathbf{p}) \prod_{j<i} \big(1-\alpha_j(\mathbf{p})\big).$$

The photometric loss and the densification are offline. Playback is projection, sorting, and blending.

Time turns each parameter into a track

$$G_i(t) = \{\mu_i(t),\, \Sigma_i(t),\, c_i(t),\, \alpha_i(t)\}.$$

DyNeRF *juggle*, the usual benchmark:

$$150 \times 336{,}568 \times 32\,\mathrm{B} = \mathbf{1.62\,GB}.$$

Those 32 bytes are μ, scale, RGBA, and a quaternion. Color in this container is diffuse RGB.

Where a folder of frames falls over

Zstandard on the raw floats reaches about 2.5×. The low bits of a mantissa look random.

A new typed array on every frame eventually pauses the tab for garbage collection.

The composite above depends on depth order, and motion invalidates that order every frame. A general sort of ∼300k keys on the main thread runs past a 16 ms frame.

The `.splat4d` file

Adam Raudonis, 2026. Choose a tolerance first. Two millimeters is the usual position bound. After that, the Rust encoder and the JavaScript player share one integer representation.

1. A property whose whole track fits in the tolerance is stored once.
2. Every other value is rounded onto a grid that enforces the tolerance.
3. A stored value changes only when the tolerance would break.
4. Time is cut into independent chunks of about a second.
5. Those integers are reordered, then compressed with Zstandard.

Still properties are stored once

A property is static when its range over the clip fits in one bin:

$$\max_t A_i(t) - \min_t A_i(t) \leq 2\epsilon_A.$$

Juggle, default tolerance. Position is static for 79% of splats, and the static section is about 4 MB. Scale and opacity are entirely static.

Color is static for 4% of splats. Color deltas are 53.5 MB of the 83.4 MB file, because that capture's colors flicker.

Quantize, then hold

Grid spacing is twice the tolerance, the same construction as SZ and ZFP.

$$\Delta = 2\epsilon, \qquad |x - \hat{x}| \le \epsilon.$$

A value on a bin edge would hop every frame. Hold keeps the previous code while the measurement stays inside its tolerance:

$$\hat{x}_t = \begin{cases} \hat{x}_{t-1} & \text{if } |x_t - \hat{x}_{t-1}| \le \epsilon, \\ Q(x_t) & \text{otherwise.} \end{cases}$$

Defaults: ± 2 mm, $\pm 4/255$ color, exact rotation, $\pm 2\%$ scale. The encoder checks every splat of every frame.

Closed groups, fetched by range

Groups of 30 frames. Each opens with a keyframe, and the rest are integer deltas. A group decodes from the static section alone.

1. `bytes=0-262143`: header and byte offsets.
2. The static section: enough to draw the first view.
3. On a scrub, the one group covering that time. Keys first, then deltas.

Published at 50 Mbps: first view in 791 ms, cold keyframe in 145 ms.

What JavaScript is for

	When	Job
Rust	once, offline	Read the frames and write `.splat4d`.
Python	around that tool	Install the binary. Serve the file today.
JavaScript	every frame, in the tab	Fetch, decode, sort, and launch the draw.

The draw shader is WGSL. JavaScript builds that pipeline, fills the buffers, and chooses the frame.

The depth sort is in the JavaScript worker, next to the decoder.

WebGPU

The GPU API for the browser. Shaders are WGSL. On this Mac, Chrome runs them on Metal.

`github.com/gpuweb/gpuweb`

WebGL let a page rasterize. A compute dispatch, and a storage buffer that dispatch can read, sit outside WebGL.

The depth order in the composite has to be rebuilt whenever the splats move. This demo still sorts in JavaScript, and it can draw through WebGL2. WebGPU is what makes that sort expressible as a compute pass in the tab, which is how a native 4D renderer already does it.

The player

A worker thread owns decode and sort, so the camera thread stays free. `fzstd` is the Zstandard decompressor, written in JavaScript.

```
const probe = await fetch(url, {
  headers: { Range: "bytes=0-262143" }
});
const rangeMode = probe.status === 206;

function unzigzag(z: number): number {
  return (z >>> 1) ^ -(z & 1);
}
```

Counting sort on 16-bit depth keys, skipped when the view has barely moved. WebGPU draws. The same player uses WebGL2 when WebGPU is absent.

Published sizes

	Raw	File	
Juggle, default tolerance	1.62 GB	83.4 MB	19.4×
Juggle, coarse, colors smoothed	1.62 GB	40.9 MB	39.5×
Best generic lossless	1.62 GB	642 MB	2.5×
Flame, default tolerance	427 MB	18.5 MB	23×
Flame, looser tolerance	427 MB	7.4 MB	58×

Lossless compression of the floats stops at 2.5×. Rounding is what reaches the other rows.

Flame is the clip I will open. At ±2 mm about 16% of its positions are static, because the fire moves. Its colors are entirely static.

Who has to open the file

A finished reconstruction often lives on the workstation that produced it. The next person may only have a browser.

A pantograph meeting a contact wire, or a pole whose cables move: the still parts are stored once, and the motion is the delta stream.

Those are sketches for a client conversation. In the room today, we have the file and the tab.

Three things to keep

The tolerance is a number in millimeters, and the file is what makes a gigabyte fit through a browser.

JavaScript fetches, decodes, sorts, and launches the draw.

The capture can leave the lab as a link.

Chrome is open.

References

- Kerbl, Kopanas, Leimkühler, Drettakis. *3D Gaussian Splatting for Real-Time Radiance Field Rendering*. ACM TOG (SIGGRAPH), 2023.
- Luiten, Kopanas, Leibe, Ramanan. *Dynamic 3D Gaussians: Tracking by Persistent Dynamic View Synthesis*. 3DV, 2024.
- Wu et al. *4D Gaussian Splatting for Real-Time Dynamic Scene Rendering*. CVPR, 2024.
- Raudonis. *splats4D: a streamable 4D Gaussian splat format with tunable error bounds.* `github.com/adamraudonis/splats4D`, 2026.
- W3C GPU for the Web Community Group. *WebGPU* and *WebGPU Shading Language*.

The byte shuffling step is the same idea used by Blosc and by the SPZ and SOGS splat packers. The drawing code is a port of Kevin Kwok's `antimatter15/splat` viewer.